



Controle de Versão com Git

# Controle de Versão com Git

---

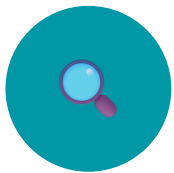
Fundamentos, Branches, GitHub e Boas Práticas

Curso Técnico de Desenvolvimento de Sistemas

*Prof. Mizael Carlos*

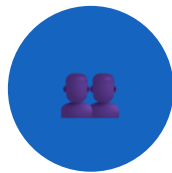
# O que é Controle de Versão?

*Controle de versão é um sistema que registra alterações em arquivos ao longo do tempo, permitindo rastrear mudanças, recuperar versões e trabalhar em equipe com segurança.*



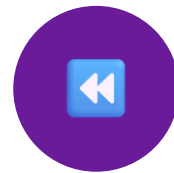
## Rastrear

Registra quem alterou, quando, o que e por quê. Todo o histórico fica disponível para consulta a qualquer momento.



## Colaborar

Múltiplos desenvolvedores trabalham simultaneamente no mesmo projeto sem sobrescrever o trabalho uns dos outros.



## Recuperar

Volte a qualquer versão anterior do projeto com um único comando. Nunca perca trabalho por engano ou experimento.



*Sem controle de versão: Projeto\_Final\_AgoraVai.zip. Com Git: histórico completo, organizado e rastreável.*

# Por que usar Git? Benefícios no Desenvolvimento

## Principais Benefícios



Nenhuma alteração é perdida — histórico completo disponível



Múltiplos devs trabalham simultaneamente no mesmo projeto



Branches para experimentar sem afetar o código principal



Integração com GitHub, GitLab, Bitbucket e Azure DevOps



Compatível com pipelines de CI/CD e deploy automatizado



Pré-requisito em praticamente toda vaga de desenvolvimento

90%

dos projetos  
profissionais  
usam Git

100  
%

das empresas  
exigem Git  
no processo

1º

sistema de  
versão mais  
usado do mundo

# Conceitos Fundamentais do Git

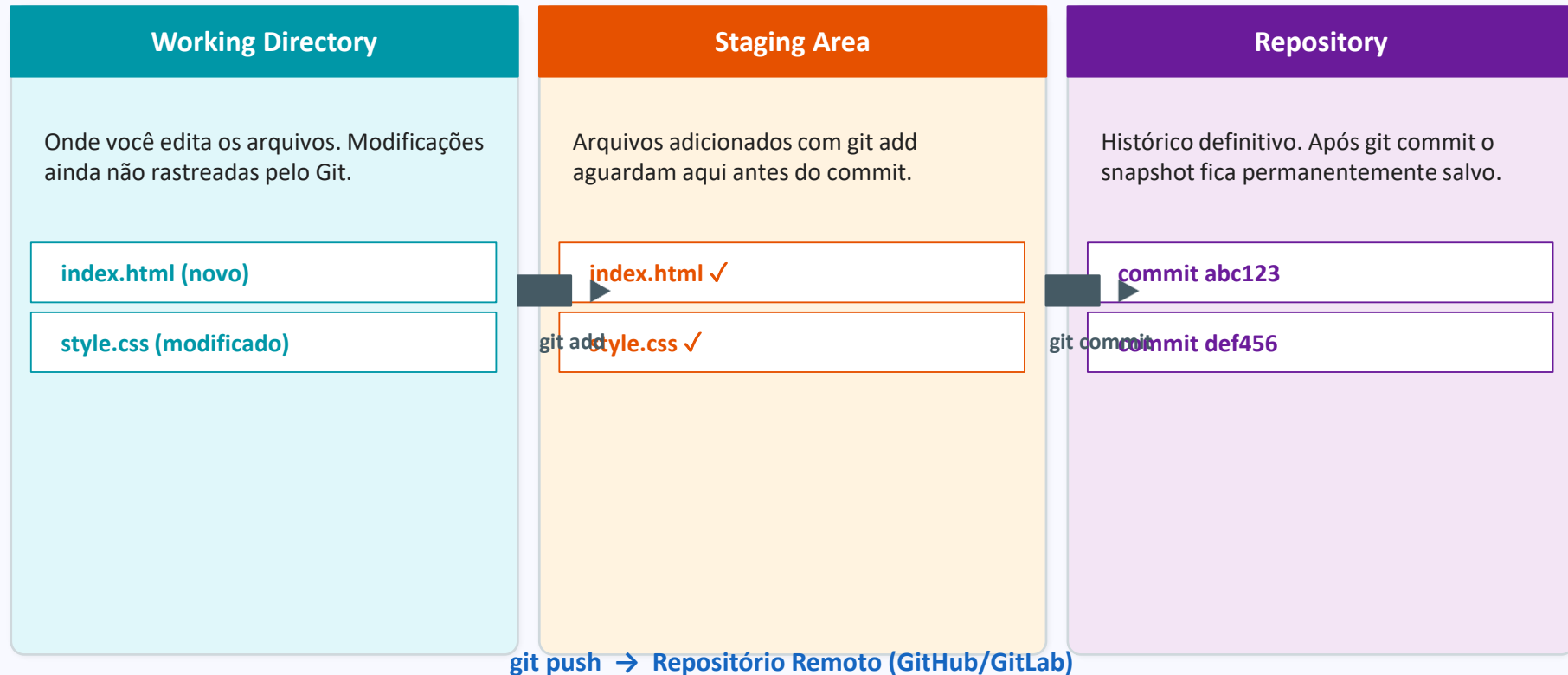
Antes de usar o Git, é essencial compreender seus conceitos-chave. Cada termo tem um papel específico no sistema.

| Conceitos Essenciais |                                   |
|----------------------|-----------------------------------|
| Repositório          | Projeto controlado pelo Git       |
| Commit               | Registro de alterações (snapshot) |
| Branch               | Linha paralela de desenvolvimento |
| Merge                | União de branches                 |
| Clone                | Cópia de repositório remoto       |
| Pull / Push          | Baixar / enviar alterações        |
| Remote               | Repositório remoto (GitHub etc.)  |

| Mais Conceitos |                                    |
|----------------|------------------------------------|
| Checkout       | Trocar de branch ou commit         |
| Staging Area   | Área temporária antes do commit    |
| HEAD           | Ponteiro para o commit atual       |
| Tag            | Marcador de versão (ex: v1.0)      |
| Stash          | Salva mudanças temporariamente     |
| Fork           | Cópia independente de um repo      |
| Rebase         | Reorganiza o histórico linearmente |

# Arquitetura do Git — Três Áreas

*O Git organiza o trabalho em três áreas distintas. Entender esse fluxo é fundamental para dominar os comandos.*



# Fluxo de Trabalho e Estados dos Arquivos

*O Git classifica cada arquivo em um estado. Entender o fluxo de estados é a chave para usar o Git com segurança.*

## Fluxo Básico de Trabalho

1

### Editar Arquivo

Modifique ou crie arquivos no Working Directory.

2

### git add

Adicione ao Stage os arquivos prontos para commit.

3

### git commit

Salve o snapshot no histórico local com uma mensagem.

4

### git push

Envie os commits para o repositório remoto.

## Estados dos Arquivos



Untracked

Arquivo novo, ainda não rastreado pelo Git.



Modified

Arquivo rastreado que foi alterado mas não está no stage.



Staged

Adicionado com git add. Pronto para o próximo commit.



Committed

Snapshot salvo com segurança no histórico do repositório.



Ignored

Arquivo listado no .gitignore. Git não o rastreia.

# Comandos Essenciais — Parte 1

*Os comandos mais usados no dia a dia. Domine esses e você já terá produtividade real com o Git.*

```
git status
```

Mostra arquivos modificados, novos e a branch atual.

```
git add arquivo.txt
```

Adiciona arquivo específico ao stage.

```
git add .
```

Adiciona todos os arquivos modificados ao stage.

```
git commit -m "msg"
```

Registra o snapshot do stage no histórico com uma mensagem.

```
Exemplo: echo "Olá Git" > index.txt → git status → git add . → git commit -m "Primeiro commit" → git log --oneline
```

```
git log
```

Exibe o histórico de commits (autor, data, mensagem).

```
git log --oneline
```

Versão resumida do histórico — uma linha por commit.

```
git diff
```

Mostra diferenças entre working directory e stage.

# Comandos Essenciais — Parte 2

*Complementando os comandos básicos: desfazendo erros, comparando versões e consultando o histórico com precisão.*

## Desfazendo Alterações

```
git commit --amend
```

```
git restore arquivo.txt
```

Descarta alterações no working directory

```
git reset HEAD~1
```

```
git reset --hard HEAD~1
```

Desfaz commit E as alterações — IRREVERSÍVEL!

## Inspecionando o Histórico

```
git log --oneline --graph
```

```
git show abc123
```

Mostra o conteúdo completo de um commit específico

```
git blame arquivo.txt
```

```
git diff HEAD~1
```

Compara o estado atual com o commit anterior



# Branches — Introdução

*Uma branch é uma linha paralela de desenvolvimento. Permite criar funcionalidades sem afetar o código em produção.*

## Por que usar Branches?

- ✓ Isole novas funcionalidades da branch principal
- ✓ Corrija bugs sem interromper o desenvolvimento
- ✓ Experimente ideias sem risco de quebrar o projeto
- ✗ Não trabalhe direto na main em projetos reais
- ✗ Não esqueça de fazer merge ao concluir a feature

## Comandos de Branch

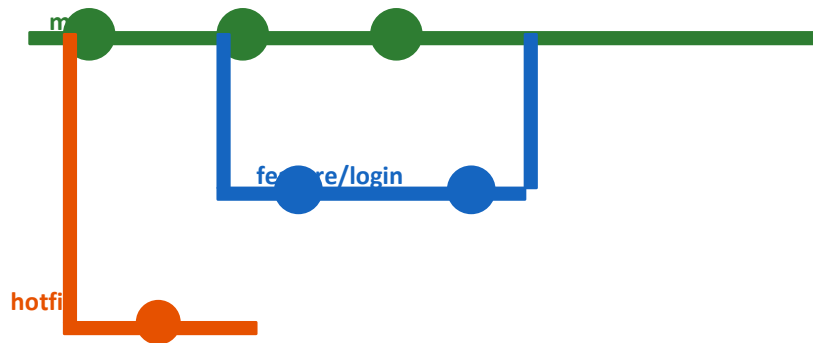
**Criar:** `git branch feature-login`

**Listar:** `git branch`

**Trocar:** `git switch feature-login`

**Criar e trocar:** `git switch -c feature-login`

## Diagrama de Branches



 Branch padrão: main (ou master em repos antigos). Cada feature merece uma branch própria.

# Merge — Unindo Branches

*Merge une as alterações de uma branch em outra. É a operação central da colaboração em equipe com Git.*

## Tipos de Merge

-  **Fast-Forward** Sem divergência. Git avança o ponteiro da branch.
-  **Merge Commit** Cria um commit de merge. Mantém o histórico real.
-  **Rebase** Move commits para o topo. Histórico linear e limpo.

## Conflito de Merge

Ocorre quando duas branches alteram a mesma linha.  
Git marca o conflito e pede decisão humana.

```
<<<<<<< HEAD
Texto A
=====
Texto B
>>>>>>> feature-login
```

## Executando um Merge

**1. Ir para main:**

```
git switch main
```

**2. Mesclar:**

```
git merge feature-login
```

**3. Resolver:**

```
# edite os arquivos em conflito
```

**4. Adicionar:**

```
git add .
```

**5. Finalizar:**

```
git commit
```

**6. Enviar:**

```
git push origin main
```

**Cancelar:**

```
git merge --abort
```

# GitHub e GitLab — Repositórios Remotos

Serviços de hospedagem Git permitem backup, colaboração, code review e integração com ferramentas de CI/CD.

| GitHub                                                                                | GitLab                                                                      | Bitbucket                                                                     |
|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <i>Maior plataforma</i>                                                               | <i>DevOps integrado</i>                                                     | <i>Ecosistema Atlassian</i>                                                   |
| Ideal para open source e portfólio profissional. Mais de 100 milhões de repositórios. | Foco em CI/CD integrado. Muito usado em empresas com pipeline automatizado. | Integração nativa com Jira e Confluence. Popular em times que usam Atlassian. |

## Comandos de Repositório Remoto

Clonar repo:

```
git clone  
https://github.com/usuario/projeto.git
```

Receber (pull):

```
git pull origin main
```

Vincular remoto:

```
git remote add origin  
https://github.com/usuario/projeto.git
```

Só baixar:

```
git fetch
```

Enviar (push):

```
git push -u origin main
```

Ver remotos:

```
git remote -v
```

# Fluxo Completo de Projeto em Equipe

*Fluxo real de trabalho colaborativo: do clone ao merge na main, passando por Pull Request e Code Review.*

1

```
git clone
```

Copia o repositório remoto para a máquina local.

2

```
git switch -c feature
```

Cria uma branch nova para a funcionalidade.

3

```
git add . && git commit
```

Adiciona e salva as alterações com mensagem clara.

4

```
git push origin feature
```

Envia a branch para o repositório remoto.

5

```
Pull Request
```

Solicita que a branch seja revisada e mesclada.

6

```
Code Review
```

O time revisa o código, aprova ou solicita ajustes.

7

```
Merge na main
```

Após aprovação, o código é integrado à branch principal.

8

```
git pull origin main
```

Todos os devs atualizam a cópia local com o merge.

# Git Flow — Estratégia de Branches

*Git Flow define branches com papéis específicos para organizar o desenvolvimento em equipe com entregas contínuas.*

**main**

*Produção*

Código pronto para produção. Sempre estável e deployável.

**develop**

*Integração*

Integração de features. Base para as branches de desenvolvimento.

**feature/nome**

*Funcionalidade*

Nova feature. Criada a partir de develop. Ex: feature/login.

**release/v1.0**

*Versão*

Preparação de release. Sai de develop, merge em main e develop.

**hotfix/nome**

*Emergência*

Correção urgente. Sai de main, merge em main e develop.

# Mensagens de Commit Profissionais

*Boas mensagens de commit tornam o histórico legível, facilitam code review e demonstram maturidade técnica.*

## Estrutura e Tipos

`tipo: descrição no imperativo`

**feat**

Nova  
funcionalidade

**refactor**

Refatoração de  
código

**fix**

Correção de bug

**test**

Adição de testes

**docs**

Documentação

**chore**

Tarefas de  
manutenção

## Boas Mensagens ✓

`feat: Adiciona validação de CPF`

`fix: Corrige cálculo de multa`

`feat: Implementa autenticação JWT`

`docs: Atualiza README com instruções`

## Mensagens Ruins ✗ — Evite!

`git commit -m "teste"`

`git commit -m "ajustes"`

`git commit -m "corrigido"`

`git commit -m "wip"`

# .gitignore — Protegendo Arquivos Sensíveis

O arquivo `.gitignore` diz ao Git quais arquivos e pastas devem ser ignorados. Essencial para não versionar dados sensíveis.

## .env e Senhas

*Nunca  
versione!*

Arquivos `.env` contêm senhas, tokens e chaves de API. Jamais devem ir para o repositório.

### Exemplos:

- `.env`
- `.env.local`
- `.env.production`
- `secrets.json`
- `*.key`
- `*.pem`

## node\_modules/ e builds

*Dependências  
e builds*

Pastas geradas automaticamente. São pesadas, regeneradas e desnecessárias no repositório.

### Exemplos:

- `node_modules/`
- `dist/`
- `build/`
- `__pycache__`
- `*.class`
- `target/`

## Arquivos do SO e IDE

*Automáticos  
e inúteis*

Criados pelo sistema operacional ou editor. Poluem o repositório e causam conflitos.

### Exemplos:

- `.DS_Store`
- `Thumbs.db`
- `.idea/`
- `.vscode/`
- `*.log`
- `*.tmp`

# Pull Requests e Code Review

*Pull Request (PR) é a solicitação para mesclar suas alterações à branch principal. Essencial em qualquer time profissional.*

## ? O que é um Pull Request?

- ✓ Solicitação para mesclar uma branch. Cria espaço para revisão antes de integrar o código ao projeto.

## ? Por que usar Pull Requests?

- ✓ Permite revisão de código antes do merge. Facilita discussão e aprovação em equipe de forma organizada.

## ? Quem revisa o código?

- ✓ Outros desenvolvedores do time ou o Tech Lead. Mínimo de 1 aprovação é boa prática em times ágeis.

## ? O que verificar no review?

- ✓ Lógica, boas práticas, segurança, cobertura de testes e aderência aos padrões do projeto.

## ? Quando fazer o merge?

- ✓ Após aprovação de pelo menos 1 revisor e com CI/CD passando — todos os testes devem estar verdes.

## ? Onde aprender mais sobre PRs?

- ✓ docs.github.com e docs.gitlab.com têm tutoriais completos com exemplos práticos e vídeos.



# Boas Práticas x Erros Comuns no Git

*Adotar boas práticas desde o início economiza tempo, evita conflitos e mantém o projeto organizado a longo prazo.*

## ✓ Boas Práticas

- Commits pequenos e frequentes com mensagens claras
- Use uma branch por funcionalidade ou correção
- Sempre faça git pull antes de começar a trabalhar
- Mantenha a main sempre funcional e pronta para deploy
- Use .gitignore para não versionar arquivos desnecessários
- Revise o código via Pull Request antes de mergear
- Use tags para marcar versões de release (v1.0, v2.0)

## ✗ Erros Comuns

- Commitar arquivos .env com senhas e tokens
- Trabalhar direto na branch main em equipe
- Usar git reset --hard sem entender o que será perdido
- Mensagens genéricas: "fix", "ajuste", "teste"
- Fazer git push --force em branches compartilhadas
- Esquecer de criar .gitignore no início do projeto
- Commitar código comentado ou arquivos de build

# O que o Mercado Exige — Conclusão

*Profissionais que dominam Git são mais contratados, colaboram melhor e entregam projetos com mais qualidade.*



## Git Básico

Commit, push, pull e branches são exigidos em 99% das vagas de desenvolvedor.



## GitHub / GitLab

Portfólio com projetos publicados é diferencial em processos seletivos.



## Pull Requests

PR e Code Review são práticas padrão em times ágeis. Saber revisar é essencial.



## Git Flow e CI/CD

Estratégias de branch são exigidas em empresas com entregas contínuas.

*Desenvolvedores que dominam Git trabalham melhor em equipe, erram menos e entregam projetos mais organizados e rastreáveis.*